

[파머완 2장]

사이킷런(Scikit-Learn) : 파이썬 기반 머신러닝 라이브러리

- 간편한 API 제공
- 다양한 머신러닝 알고리즘 지원
- 검증된 라이브러리
- 아나콘다(Anaconda) 기본 포함

붓꽃 데이터 세트(Iris Dataset): 머신러닝에서 가장 많이 사용되는 분류 예제 데이터

Feature: Sepal과 Petal의 Length 및 Width (4가지 속성)

Label: 붓꽃의 품종 (Setosa=0, Versicolor=1, Virginica=2)

지도학습(Supervised Learning):

- 정답(label)이 있는 데이터 학습 후 새로운 데이터 예측
- Train 데이터와 Test 데이터로 나누어 모델 평가

분류 (Classification)

- 지도학습 중 하나
- 데이터를 특정 클래스로 분류하는 머신러닝 문제

붓꽃 데이터 로드: `load_iris()`

Feature와 Label 분리

Pandas DataFrame 변환 → 데이터 확인 및 시각화

`train_test_split()` → 학습 데이터와 테스트 데이터로 분리

`test_size = 0.2` → 20%는 테스트 데이터로 사용

`random_state = 11` → 실행할 때마다 동일 데이터 분리 유지

사용된 알고리즘: 의사 결정 트리 (Decision Tree)

`DecisionTreeClassifier(random_state=11)` → 모델 생성

`.fit(X_train, y_train)` → 모델 학습

`predict(X_test)` → 학습된 모델 사용하여 테스트 데이터의 품종 예측

`accuracy_score(y_test, pred)` → 정확도 측정(모델 평가)

Estimator: 사이킷런에서 머신러닝 알고리즘을 구현한 모든 모델

지도학습 알고리즘을 구현한 클래스

- Classifier: `DecisionTreeClassifier`, `SVC` 등

- Regressor: `LinearRegression`, `Ridge` 등

Estimator는 모든 지도학습 모델에서 `fit()`과 `predict()` 사용 가능

하이퍼파라미터 튜닝, 교차 검증 등에서 Estimator를 입력값으로 사용

Estimator의 핵심 method

- `fit()`: 학습 수행, 데이터를 학습하여 모델을 생성하는 과정

- `predict()`: 예측 수행, 학습된 모델 바탕으로 새로운 데이터에 대한 예측 수행

지도/비지도 학습 fit() 차이

- 지도학습 `fit(X_train, y_train)`: 모델을 학습 데이터로 훈련
  - 비지도 학습 `fit(X)`: 데이터의 패턴 분석하여 변환 구조 학습
  - 비지도 학습 `transform(X)`: 학습된 구조 기반으로 데이터 변환
  - 비지도 학습 `fit_transform(X)`: `fit()`과 `transform()`을 동시 수행
- 비지도 학습에서는 학습 위한 레이블이 없으므로 `fit()`이 데이터 구조 파악하는 역할  
이후 `transform()` 사용하여 데이터 변환

사이킷런 주요 모듈

- 예제 데이터

sklearn.datasets

- 피처 처리

sklearn.preprocessing

sklearn.feature\_selection

sklearn.feature\_extraction

- 피처 처리 & 차원 축소

sklearn.decomposition

- 데이터 분리, 검증&파라미터 튜닝

sklearn.model\_selection

- 평가

sklearn.metrics

- ML 알고리즘

sklearn.ensemble

sklearn.linear\_model

sklearn.naive\_bayes

sklearn.neighbors

sklearn.svm

sklearn.tree

sklearn.cluster

- 유틸리티

sklearn.pipeline

## 내장 예제 데이터 세트

- 사이킷런에서 외부 데이터 다운로드 없이 제공하는 머신러닝 연습용 데이터 세트
- 분류, 회귀, 클러스터링 등
- datasets 모듈 통해 로드 가능

## 내장 데이터 세트 종류

- 분류, 회귀 연습용 데이터

load\_boston() 회귀, 보스턴 집값 데이터

load\_breast\_cancer() 분류, 위스콘신 유방암 데이터

load\_diabetes() 회귀, 당뇨병 데이터

load\_digits() 분류, 손글씨 숫자(0-9) 이미지 데이터

load\_iris() 분류, 붓꽃 품종 데이터

fetch 계열 대용량 데이터는 인터넷에서 다운 후 사용 가능

## 분류 및 클러스터링용 표본 데이터 생성기

- make\_classification(): 분류 문제용 가상의

## 데이터 생성(노이즈 포함 가능)

- make\_blobs(): 클러스터링 문제용 데이터 생성

## 내장 데이터 세트의 구조

- 대부분의 내장 데이터 세트는 파이썬

딕셔너리와 유사한 bunch 객체로 반환

주요 key 구성 요소

- data 피쳐 데이터
- target 정답 데이터
- target\_names 레이블 이름
- features\_names 피쳐 이름 리스트
- DESCR 데이터 세트 설명

Bunch 객체는 파이썬 딕셔너리와 유사한 데이터 구조

model\_selection 모듈

- 머신러닝 모델 평가 및 하이퍼파라미터 튜닝을 위한 기능 제공
- 주요 기능

train\_test\_split(): 학습/테스트 데이터 분리

cross\_val\_score(): 교차 검증

GridSearchCV, RandomizedSearchCV: 하이퍼파라미터 튜닝

학습 데이터만으로 모델을 평가

→ 모델이 이미 학습 데이터의 정답을 앎

→ 정확도 100% (과적합 문제)

⇒ 학습 데이터와 테스트 데이터를 분리해야 됨

⇒ train\_test\_split() 이용

train\_test\_split() 주요 파라미터

- test\_size: 테스트 데이터 비율
- train\_size: 학습 데이터 비율(보통 생략)
- shuffle: 데이터 섞을지의 여부(기본값 True)
- random\_state: 실행할 때마다 동일한 덩어리 분할 유지

데이터 양이 적을 때 테스트 데이터 비율 너무 높이면 안 됨

교차 검증(Cross Validation)

- 과적합 방지 위해 학습 데이터 여러 번 나누어 모델 평가 방법
- 특정 테스트 데이터에만 편향되는 것 방지
- 고정된 학습/테스트 데이터만으로 평가하면 특정 데이터에 과적합 위험
- 교차 검증 통해 데이터 분할 활용하여 일반화 성능 더 정확히 평가

K-Fold 교차 검증

- 가장 보편적인 교차 검증
- 데이터를 K개의 Fold로 나누어 K번 학습 및 평가
- 각 Fold마다 학습/검증 데이터 번갈아 사용
- 최종 결과는 K번의 검증 결과를 평가하여 모델 성능 평가

K-Fold 교차 검증 프로세스

- 1) 데이터 전체를 K개로 분할
- 2) 첫 번째 Fold를 검증 데이터, 나머지를 학습 데이터로 사용하여 모델 학습 및 평가
- 3) 두 번째 Fold를 검증 데이터, 나머지를 학습 데이터로 변경하여 반복
- 4) 총 K번 반복 후, 평균 정확도를 최종 성능으로 평가

→ K번 반복 수행해야 하므로 계산량 증가

→ K번 학습&평가 반복해야 하므로 속도 저하 가능

#### K-Fold 교차 검증의 문제점

- 랜덤 분할로 인해 특정 Fold에서 특정 클래스가 포함되지 않을 가능성 있음
- 데이터 클래스 비율 유지 안 됨
- 불균형 데이터에서 심각한 편향 발생

#### Stratified K-Fold 교차 검증

- 각 Fold에 클래스 비율을 유지하도록 데이터 분할
  - split() method에서 레이블 데이터도 반드시 입력해야 함
- 불균형 데이터에서도 모델이 모든 클래스를 학습할 수 있도록 보장됨

#### cross\_val\_score()

- 교차 검증을한 번에 수행할 수 있도록 도와주는 사이킷런 API
- K-Fold 또는 StratifiedKFold와 달리 자동으로 교차 검증 수행
- 모델 학습(fit) -> 예측(predict) -> 평가(evaluate)를 자동으로 수행
- 평균 정확도 등 다양한 성능 지표 반환
- 분류에서는 자동으로 StratifiedKFold 사용
- 회귀에서는 K-Fold 사용



`cross_val_score()` 주요 파라미터

- estimator: 사용 모델
- x: 피쳐 데이터
- y: 레이블 데이터
- scoring: 평가 지표(accuracy, precision 등)
- cv: 교차 검증 폴드 개수

`cross_val_score()` 내부 동작 방식

- cv 설정으로 n개의 Fold 생성
- 각 Fold마다 수행
  - 1) 학습 데이터와 검증 데이터로 분할
  - 2) 모델 학습(fit)
  - 3) 예측 수행(predict)
  - 4) 성능 평가(accuracy\_score)
  - 5) n번 반복 후, 최종 평균 정확도 반환

→ `cross_val_score()`는 분류 문제에서 자동으로 StratifiedKFold를 사용하여 클래스 비율 유지하면서 Fold 생성

`cross_val_score()`: 단일평가 지표만 반환(accuracy, roc\_auc 등)

`cross_validate()`: 여러 개의 평가 지표 반환 가능(test\_accuracy, test\_precision 등)

⇒ 교차 검증 필요할 땐 `cross_val_score()`, 여러 지표 필요하면 `cross_validate()`

## GridSearchCV

- 하이퍼파라미터 튜닝을 자동화하는 사이킷런 API
- 하이퍼파라미터를 조합하여 최적의 조합 찾음
- 교차 검증 통해 성능 평가
- 사용자가 튜닝할 하이퍼파라미터를 미리 지정하면 모든 조합 테스트, 가장 좋은 성능 찾음
- 교차 검증과 함께 수행, 과적합 방지
- 최적의 조합 자동으로 찾아 다시 학습(refit=True)

## 하이퍼파라미터

- 머신러닝 모델이 학습하기 전에 사용자가 직접 설정하는 값
- e.g.) DecisionTree의 하이퍼파라미터: max\_depth, min\_samples\_split
- 모델 성능 최적화하려면 적절한 하이퍼파라미터 찾아야 함

## GridSearchCV의 주요 파라미터

- estimator: 모델
- param\_grid: 테스트할 하이퍼파라미터 조합(딕셔너리 형태)
- scoring: 평가 지표
- cv: 교차 검증 Fold 개수
- refit: 최적의 하이퍼파라미터로 학습된 모델 저장 여부(True 기본값)

## GridSearchCV 주요 속성

- best\_params\_: 최적의 하이퍼파라미터 조합
- best\_score\_: 최고 성능(교차 검증 평균 정확도)
- best\_estimator\_: 최적의 모델(이미 학습 완료)

## 데이터 전처리

- 머신러닝 모델의 성능을 높이기 위해 데이터를 정제하고 변환하는 과정
- Garbage In, Garbage Out: 잘못된 데이터 입력하면 결과도 좋지 않음
- 사이킷런의 머신러닝 알고리즘은 반드시 정제된 데이터 입력

## 데이터 전처리의 주요 과정

- 1) 결손값 처리
- 2) 문자열 처리
- 3) 데이터 스케일링
- 4) 특성 선택

## 결손값 처리

- Null 값이 포함된 경우 사이킷런 모델 학습 불가능
- 결손값 처리 방법
  - 1) 삭제: 결손값이 많거나 중요하지 않은 feature 삭제
  - 2) 대체: 결손값이 적거나 중요한 경우 평균, 중앙값 등으로 대체

## 문자열 데이터 처리

- 사이킷런의 머신러닝 알고리즘은 문자열 값을 입력값으로 허용 안 함
- 모든 문자열값은 숫자형으로 변환(인코딩)해야 함
- 카테고리형 feature: 코드 값으로 변환
- 텍스트형 feature: 벡터화 또는 삭제(필요하지 않다면)

## 데이터 인코딩

### 1) 레이블 인코딩

- 문자열 값을 숫자로 변환
- LabelEncoder 클래스 사용하여 fit() → transform() 순서로 변환
- 숫자의 크기가 의미를 가질 수 있으니 선형 회귀 모델에는 적용 않는 것이 좋음
- 트리 계열 알고리즘에서는 별 문제 없음

### 2) 원-핫 인코딩

- 레이블 인코딩의 문제점 해결
- 고유값을 새로운 feature로 추가하고, 해당 값에만 1을 부여
- OneHotEncoder 사용 → 입력값은 반드시 2차원 배열
- 변환 결과는 희소 행렬 형태이므로 toarray() 사용, 밀집 행렬로 변환

### 3) 판다스 get\_dummies()

- 원-핫 인코더와 달리 바로 변환 가능
- 숫자로 변환할 필요 없이 문자열 데이터를 바로 원-핫 인코딩 가능

## 피쳐 스케일링

- 서로 다른 변수의 값 범위를 일정 수준으로 맞추는 작업
- 대표적 방법: 표준화, 정규화

## 표준화

- 데이터를 평균이0, 분산이1인 가우시안 정규 분포로 변환
- 변환된 값 =  $(x - \text{평균}) / (\text{표준편차})$
- 일부 알고리즘에서는 표준화가 성능 향상에 중요하게 작용
- `StandScaler()`

## 정규화

- 서로 다른 feature의 크기를 통일하기 위해 값을 최소 0-1 범위로 변환
- 변환된 값 =  $(x - \text{최솟값}) / (\text{최댓값} - \text{최솟값})$
- 데이터가 가우시안 정규 분포 따르지 않으면 Min-Max Scaling 적용 가능
- `MinMaxScaler()`

## 벡터 정규화

- 사이킷런의 정규화 모듈은 선형대수 개념의 정규화 적용
- 개별 벡터의 크기를 맞추기 위해 모든 feature 벡터의 크기로 나눠줌

학습 데이터와 테스트 데이터의 스케일링 변환 시 유의점

- Scaler 객체를 이용해 fit(), transform(), fit\_transform()을 사용할 때 학습데이터, 테스트 데이터의 처리 방식 다름

fit()

- 데이터 변환 위한 기준 정보 설정 (최대/최솟값 등)

transform()

- 설정된 정보를 기반으로 데이터를 변환

fit\_transform()

- fit()과 transform() 동시에 수행

학습데이터

- fit() → transform() 적용

테스트 데이터

- transform()만 적용

fit() 잘못 적용하면

- 학습 데이터와 테스트 데이터가 다른 기준으로 변환
- 잘못된 변환으로 인해 모델 예측 성능 저하

⇒ 가능하면 전체 데이터를 먼저 스케일링한 후 학습/테스트 데이터로 분리

⇒ 불가능하다면 학습 데이터에만 fit() 적용 후, 테스트 데이터는 transform()만 수행

[파머완 3장]

머신러닝 과정

- 1) 데이터 가공/변환
- 2) 모델 학습/예측
- 3) 평가

→ 모델 성능 평가 방법은 분류와 회귀에 따라 다름

분류 성능 평가 지표 (특히 이진 분류)

- 정확도
- 오차 행렬
- 정밀도
- 재현율
- F1 스코어
- ROC AUC

정확도의 한계

- 직관적인 지표지만, 데이터 구성에 따라 모델 성능 왜곡 가능성 있음
- 단순한 규칙만으로도 높은 정확도 기록 가능

오차 행렬

- 이진 분류 모델의 성능을 평가하는 주요 지표
- 학습된 분류 모델이 얼마나 정확하게 예측했는지 4가지 값으로 나누어 표현

오차 행렬 4가지 값

- TN (True Negative): 실제 0, 예측 0
- FP (False Positive): 실제 0, 예측 1
- FN (False Negative) → 실제 1, 예측 0
- TP (True Positive) → 실제 1, 예측 1
- 단순 예측 모델이라도 정확도 높고 성능 엉망일 수 있음

오차행렬 활용한 정확도 계산

- (정확도) =  $(TP+TN) / (TP+TN+FP+FN)$
- 이진 분류에서는 정확도만으로 모델의 성능 판단하면 안 됨

소수의 positive(1) 데이터가 중요한 경우 많음

정확도가 높더라도 모델이 의미 있는 예측을 못할 수 있음

→ 정밀도와 재현율이 더 중요한 경우 많음

⇒ 불균형한 데이터에서는 정확도 대신 정밀도와 재현율 함께 고려

정밀도

- 모델이 Positive(양성, 1)로 예측한 것 중 실제로 Positive인 비율
- Precision =  $TP / (TP+FP)$
- FP, 잘못된 양성 예측을 줄이는 것 중요

재현율

- 실제 Positive(양성, 1) 중에서 모델이 Positive로 제대로 예측한 비율
- Recall =  $TP / (TP+FN)$
- FN, 놓친 양성 예측 줄이는 것 중요



### 재현율 중요한 경우

- 실제 Positive 데이터 놓치면 안 되는 경우
- 암 진단, 금융 사기 탐지

### 정밀도 중요한 경우

- 실제 Negative 데이터를 Positive로 잘못 예측하면 안 되는 경우
- 스팸 필터링, 광고 추천

### 정밀도, 재현율 한꺼번에 계산하는 함수

- `accuracy_score()` : 정확도
- `precision_score()`: 정밀도
- `recall_score()`: 재현율
- `confusion_matrix()`: 오차 행렬
- `get_clf_eval()`: 모든 평가 지표 한 번에 출력

### 정밀도-재현율 트레이드 오프

- 정밀도와 재현율은 반비례 관계
- 분류 모델의 결정 임계값(threshold) 조정하면 정밀도, 재현율 조절 가능
- 임계값 낮추면 재현율 증가, 정밀도 감소
- 임계값 높이면 정밀도 증가, 재현율 감소

### 예측값

- 사이킷런의 `predict_proba()` 메서드는 각 클래스(0과 1)의 예측 확률 반환
- 예측 확률 0.5 이상이면 1
- 예측 확률 0.5 미만이면 0

상황에 따라 임계값 선택

- 암 진단, 금융 사기 탐지: 재현율 중요, 임계값 낮춤
- 스팸 필터, 광고 추천 모델: 정밀도 중요, 임계값 높임

정밀도, 재현율 문제점

- 단독으로 모델 성능 평가 어려움
- 한쪽만 극단적으로 높이는 것은 현실적 의미 없음
- 비즈니스 환경에 따라 균형 있는 수치 유지 중요

정밀도 100% 만들기

- 확실한 경우만 Positive 예측, 나머지 모두 Negative
- 실제 Positive 중 놓친 경우 (FN) 너무 많음
- 재현율 매우 낮아져서 모델 쓸모 없음

재현율 100% 만들기

- 모든 데이터 Positive 예측
- FP 너무 많음
- 정밀도 매우 낮아짐, 모델 신뢰도 하락

F1 스코어

- 정밀도 재현율 결합 지표
- 두 지표 극단적 차이 나면 낮은 값 가짐
- $F1 = 2 \times ((\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}))$

## F1 스코어 특성

- 정밀도 재현율 중 하나가 낮으면 F1 스코어 낮아짐
- 두 값이 비슷할 때 F1 스코어 상대적으로 높아짐
- `f1_score()` API 사용
- 임계값 조정하면서 적절한 균형 찾아야 함]

## ROC 곡선

- 이진 분류의 성능 평가 그래프
- FPR 과 TPR 의 관계 나타냄
- X축: FPR, Y축: TPR
- `roc_curve()` 이용, `predict_proba()`의 positive 확률 활용
- 

## TPR(민감도)

- $TP / (TP + FN)$
- 재현율임

## TNR(특이성)

- $TN / (FP + TN)$
- 높을수록 Negative 예측 정확함

## FPR

- 1-특이성
- 낮을수록 Positive 예측 적음

## AUC

- ROC 곡선 아래 면적
- 클수록(1에 가까울수록 분류 성능 뛰어남)
- 일반적으로 0.8 이상이면 좋은 모델